

Auditor Bancario VCT — Informe público: código fuente explicado

Versión: 2.12.11 · Público — apto para GitHub, Patreon y NotebookLM

Autor: Angel del Valle Calero (calero89) · © 2026 · Vanilla Center Trust [VCT]

Sin datos de infraestructura, tokens ni IDs de servidor

1. ¿Qué es este proyecto en código?

Bot Python 3 + discord.py con arquitectura modular:

```
Auditor_Bancario_bot.py      # Entrada
└─ vct_auditor/              # Paquete principal
  └─ config.py                # Reglas de juego (constantes)
  └─ bot_core.py              # Bot + eventos Discord
  └─ storage.py               # Persistencia JSON
  └─ commands/                # ~89 slash commands
  └─ [módulos de dominio]     # economía, FS22, crimen, tiendas...
```

2. Arranque en 4 pasos

python

1. Cargar token desde entorno (nunca hardcodeado)

```
DISCORD_BOT_TOKEN = _cargar_variable_env("DISCORD_BOT_TOKEN")
```

2. Crear bot con intents

```
intents = discord.Intents.default()
intents.members = True
intents.message_content = True
```

3. Al conectar: cargar JSON, registrar comandos, sync slash

```
async def setup_hook(self):
    self.cargar_datos()
    register_commands(self.tree)
    await self.tree.sync(guild=guild)
```

4. Ejecutar

```
bot.run(DISCORD_BOT_TOKEN)
```

3. Constantes de economía (config.py)

Todas las reglas de juego viven en un solo archivo:

```
python
IMPUESTO_TRANSACCION = 0.05    # 5 % transferencias legales
IMPUESTO_SEMANAL = 0.20        # Lunes sobre cartera
ATRACO_EXITO_CHANCE = 0.35     # 35 % atraco en equipo
CONTRABANDO_INSPECCION_CHANCE = 0.30
TRABAJOS_FS22 = {
    "cosecha": "Cosecha",
    "hilarar": "Hilarar",
```

... 36 tipos

```
}
```

Ventaja de diseño: cambiar economía sin tocar lógica de comandos.

4. Persistencia segura (storage.py)

Versión didáctica (autocontenida). En producción se añaden carpeta destino, backup .ultimo_ok y reintentos ante PermissionError. El mismo fragmento vive en ejemplos/06_persistencia_json_atmica.py.

```
python
import json
import os
import time

def _fsync_directorio(ruta: str) -> None:
    directorio = os.path.dirname(os.path.abspath(ruta)) or "."
    try:
        fd = os.open(directorio, os.O_RDONLY)
    except OSError:
        return
    try:
        os.fsync(fd)
    finally:
        os.close(fd)

def guardar_json_atmico(ruta: str, datos) -> None:
    """Escribe datos en ruta vía temporal + replace + fsync."""
    temporal = f"{ruta}.{os.getpid()}.{time.time_ns()}.tmp"
    temporal_completo = False
```

```
try:
with open(temporal, "w", encoding="utf-8") as f:
    json.dump(datos, f, indent=4, ensure_ascii=False)
    f.flush()
    os.fsync(f.fileno())
    temporal_completo = True
    os.replace(temporal, ruta) # Atómico en POSIX/Windows
    _fsync_directorio(ruta)
except Exception:
    if not temporal_completo and os.path.exists(temporal):
        try:
            os.remove(temporal)
        except OSError:
            pass
        raise
```

Si replace falla tras escribir el temporal completo, no se borra el .tmp: queda para recuperación manual. Solo se eliminan temporales parciales (serialización interrumpida).

Cada cuenta de jugador es un objeto en banco_vct.json indexado por ID Discord.

5. Lock anti double-spend (banco_sync.py)

El fsync del JSON no debe bloquear el event loop ni otros slash (límite Discord 3 s). Tras mutar en memoria se hace snapshot (también si el comando falla) y se persiste en un hilo; los guardados se serializan para que una foto antigua no sobrescriba una más reciente. Las transacciones anidadas reutilizan el mismo lock externo (sin deadlock).

```
python
import asyncio
import copy
from collections.abc import Awaitable, Callable
from contextlib import asynccontextmanager
from contextvars import ContextVar

PostCommit = Callable[[], Awaitable[None]]
_profundidad = ContextVar("_profundidad", default=0)
_post_commit_actual = ContextVar("_post_commit_actual", default=None)

def al_confirmar_persistencia(callback: PostCommit) -> None:
    callbacks = _post_commit_actual.get()
    if callbacks is None:
        raise RuntimeError("requiere transaccion_banco activa")
    callbacks.append(callback)
```

```
@asynccontextmanager
async def transaccion_banco(*, persistir: bool = True):
    if _profundidad.get():
        token = _profundidad.set(_profundidad.get() + 1)
    try:
        yield bot
    finally:
        _profundidad.reset(token)
    return

callbacks: list[PostCommit] = []
async with bot._banco_lock:
    depth_token = _profundidad.set(1)
    pc_token = _post_commit_actual.set(callbacks)
    snapshot = None
    try:
        yield bot
    finally:
        if persistir:
            snapshot = copy.deepcopy(bot.banco)
            _post_commit_actual.reset(pc_token)
            _profundidad.reset(depth_token)
        if snapshot is not None:
            async with bot._banco_persist_lock:
                await asyncio.to_thread(guardar_json_atómico, ruta_banco(), snapshot)
    for callback in callbacks:
        await callback()
```

Todos los slash commands se envuelven al registrarse:

```
python
```

commands/__init__.py

```
def register_commands(tree):
    bank.setup(tree)
    crime.setup(tree)

...

envolver_arbol_comandos(tree) # ← último paso
```

Los callbacks de discord.ui.View (botones/menús persistentes) no pasan por CommandTree, así que las tiendas que mutan saldos deben entrar explícitamente en transaccion_banco() dentro del handler.

6. Comando slash típico

```
python
@tree.command(name="depositar", description="Mover dinero de cartera a caja fuerte")
async def depositar(interaction: discord.Interaction, monto: int):
    await interaction.response.defer(ephemeral=True)
    bot = get_bot()
    bot.check_user(interaction.user.id)    # Crea cuenta si no existe
    datos = bot.banco[str(interaction.user.id)]
    if monto <= 0:
        await interaction.followup.send(" El monto debe ser positivo.", ephemeral=True)
        return
    if datos["balance"] < monto:
        await interaction.followup.send(" Saldo insuficiente.", ephemeral=True)
        return
    datos["balance"] -= monto
    datos["vault"] += monto

    async def confirmar_guardado():
        await interaction.followup.send(f" Guardados {monto} €", ephemeral=True)

    al_confirmar_persistencia(confirmar_guardado)
```

(En producción el guardado ocurre dentro del lock automático; las respuestas de éxito que prometen persistencia se envían después de confirmar el snapshot.)

7. Autocompletado FS22 (límite Discord 25)

Discord no admite más de 25 choices estáticas. Solución:

```
python
def choices_autocomplete_trabajos_fs22(current: str) -> list[tuple[str, str]]:
    busqueda = normalizar(current)
    candidatos = filtrar(TRABAJOS_FS22, busqueda)
    return candidatos[:25] # Tope API Discord

@tree.command(name="registrar_contrato_fs22", ...)
@app_commands.autocomplete(tipo=_autocomplete_tipo_fs22) # @tree.command PRIMERO
async def registrar_contrato_fs22(interaction, tipo: str):
    ...
```

8. Guard de arresto en tiendas

```
python
async def bloquear_si_arrestado(interaction, banco) -> bool:
    if interaction.user.bot:
        return False
    uid = str(interaction.user.id)
    if uid not in banco or not esta_arrestado(banco[uid]):
        return False
    try:
        if interaction.response.is_done():
            await interaction.followup.send(MENSAJE_BLOQUEO_ARRESTO, ephemeral=True)
        else:
            await interaction.response.send_message(MENSAJE_BLOQUEO_ARRESTO, ephemeral=True)
    except discord.HTTPException:
        pass
    return True
```

Se llama al inicio de cada botón de catálogo persistente.

9. Persecución y prisión (flujo)

```
/robar (éxito) → NRD + registro persecución 24h
↓
/perseguir (víctima) → minijuego
↓ captura
arrestar_jugador() → incauta NRD/inventario, rol Recluso
↓
publicar_expediente_penitenciario() → canal prisión
↓
/pagar_fianza_vct o /trabajos_comunitarios ×5
↓
publicar_limpieza_expediente() → libertad + antecedentes borrados
```

10. Contrabando FS22 (v2.9.6)

```
python
CONTRABANDO_FS22_PRODUCTOS = {
    "trigo_negro": ("Trigo fuera de contrato", "agricultura", 8000, 18000, 4, 7),
```

nombre, sector, nrd_min, nrd_max, rep_min, rep_max

```
}
```

```
if random.random() < 0.30:
```

Inspección: multa legal + antecedente

```
else:
```

```
ganancia = random.randint(nrd_min, nrd_max)
```

```
datos["dinero_negro"] += ganancia
```

```
modificar_reputacion(datos, criminal=rep_ganada)
```

11. Tiendas persistentes (shop_views.py)

```
python
```

```
class TiendaLegalView(ui.View):
```

```
def __init__(self, banco):
```

```
super().__init__(timeout=None) # Persistente
```

```
self.banco = banco
```

```
@ui.button(label="Comprar", custom_id="vct_tienda_legal_comprar")
```

```
async def comprar(self, interaction, button):
```

```
async with transaccion_banco():
```

```
if await bloquear_si_arrestado(interaction, self.banco):
```

```
return
```

validar saldo → descontar → conceder item...

custom_id fijo permite que el bot recuerde botones tras reinicio.

12. Monetización dual

Canal	Módulo	Resultado
Ko-fi webhook	kofi.py	Rol Ciudadano / Socio Preferente
Discord SKU	monetization.py	Rol Socio VCT + /verificar_socio_vct

```
python
```

```
SKU_SOCIO_VCT = int(_cargar_variable_env("DISCORD_SKU_SOCIO_VCT_ID", "0") or "0")
```

```
async for ent in bot.entitlements(exclude_ended=True):
```

```
if ent.sku_id == SKU_SOCIO_VCT and ent.user_id == miembro.id:
```

```
await otorgar_rol_socio_vct(miembro)
```

La verificación cruza siempre SKU y propietario de la suscripción; un entitlement activo de otro usuario nunca debe conceder el rol al miembro que ejecuta el comando.

13. Más ejemplos en el repositorio

Carpeta ejemplos/:

Archivo	Tema
01_economia_constantes.py	Tasas y FS22
02_carga_variables_entorno.py	kofi.env
03_guard_arresto_tiendas.py	Bloqueo reclusos
04_fs22_autocomplete.py	Autocompletado
05_lock_transacciones_banco.py	Lock async
06_persistencia_json_atomica.py	JSON atómico (tmp + replace + fsync archivo/direct

14. Stack y dependencias principales

- discord.py 2.x — API Discord
 - aiohttp — webhook Ko-fi
 - JSON local — sin base de datos externa
 - Servicio 24/7 en Linux
-